

Práctica de laboratorio

Monitor interactivo del RP2040 y exploración del SIO

Materia	Elementos Programables I
Plataforma	Raspberry Pi Pico - RP2040
Duración	4 horas
Modalidad	Individual

1. Objetivo

Explorar el funcionamiento interno de las entradas y salidas digitales del RP2040 mediante un monitor interactivo de comandos, relacionando las operaciones realizadas sobre los GPIO con los registros del bloque SIO (Single-cycle I/O).

Al finalizar la práctica, el estudiante deberá ser capaz de:

- Compilar y cargar un proyecto del Pico SDK en una Raspberry Pi Pico.
- Utilizar una consola de comandos sobre la tarjeta.
- Explicar la función de GPIO_IN, GPIO_OUT y GPIO_OE.
- Interpretar y utilizar máscaras de bits del tipo $1u \ll n$.
- Relacionar sio_hw->gpio_set, gpio_clr, gpio_togl, gpio_in y gpio_oe con registros hardware.
- Identificar la relación entre sio_hw_t y el mapa de registros SIO del RP2040.
- Modificar el monitor agregando un comando propio.

2. Material proporcionado y hardware

El profesor proporcionará un proyecto funcional. La primera parte consiste en utilizarlo como una caja negra; después se revisará el código para entender cómo cada comando accede al hardware.

```
help
on
off
toggle
read
sio
mask
```

- Raspberry Pi Pico original con RP2040.
- LED integrado en GPIO25.
- Pulsador externo conectado a GPIO13 y 3.3 V.
- Resistencia pull-down interna habilitada por software.
- Tera Term, terminal serial de Visual Studio Code o equivalente.

3. Desarrollo de la práctica

Parte 1. Compilación y puesta en funcionamiento

1. Abra el proyecto proporcionado en Visual Studio Code.
2. Compile utilizando el Raspberry Pi Pico SDK.
3. Verifique la generación del archivo Monitor_REPL_RP2040.uf2.
4. Cargue el archivo en la Pico mediante BOOTSEL.
5. Abra el puerto serial con Tera Term, Visual Studio Code u otra terminal equivalente.
6. Verifique que aparezca el prompt RP2040>.
7. Ejecute el comando help.

RP2040> help

Evidencia 1. Captura del monitor funcionando y mostrando el comando help.

Parte 2. Exploración libre del monitor

Antes de revisar el código fuente, experimente con los comandos y registre qué ocurre físicamente y qué informa la terminal.

Comando	¿Qué ocurrió físicamente?	¿Qué mostró la terminal?
on		
off		
toggle		
read		
sio		
mask		

En esta etapa responda primero “¿qué hace el sistema?” y deje para después “¿cómo lo hace?”.

Parte 3. Exploración de GPIO_OUT

RP2040> off

RP2040> sio

Registre GPIO_OUT. Después ejecute:

RP2040> on

RP2040> sio

Compare ambos valores e identifique qué bit cambia. El LED integrado corresponde a GPIO25.

RP2040> mask

Relacione GPIO25, el bit 25, la expresión $1u \ll 25$ y el valor hexadecimal 0x02000000.

Evidencia 2. Captura de GPIO_OUT con el LED apagado y encendido, señalando el bit que cambia.

Parte 4. Exploración de GPIO_IN

RP2040> read

Ejecute primero con el botón libre y después manteniéndolo presionado. Repita con el comando sio e identifique el bit correspondiente a GPIO13.

Evidencia 3. Captura de la lectura del GPIO13 con el botón libre y presionado.

Parte 5. Interpretación de GPIO_OE

RP2040> sio

Observe GPIO_OE y analice específicamente los bits 25 y 13. Interprete:

GPIO_OE bit = 1 -> GPIO habilitado como salida

GPIO_OE bit = 0 -> salida deshabilitada; puede utilizarse como entrada

Registro	Función
GPIO_IN	Estado lógico observado en los GPIO
GPIO_OUT	Valor almacenado para las salidas
GPIO_OE	Determina cuáles GPIO están habilitados como salida

Parte 6. Revisión guiada del código

Abra main.c y localice las siguientes líneas. Para cada una, escriba en su hoja de evidencias qué operación de hardware representa.

```
#define LED_PIN 25
#define BUTTON_PIN 13
#define LED_MASK (1u << LED_PIN)
#define BUTTON_MASK (1u << BUTTON_PIN)
```

Explique qué significa `1u << LED_PIN` y por qué permite seleccionar un GPIO.

Encender una salida

```
sio_hw->gpio_set = LED_MASK;
```

Relacione `sio_hw`, `gpio_set`, `GPIO_OUT_SET`, `LED_MASK` y el bit 25.

Apagar una salida

```
sio_hw->gpio_clr = LED_MASK;
```

Identifique la diferencia entre `gpio_set` y `gpio_clr`.

Invertir una salida

```
sio_hw->gpio_togl = LED_MASK;
```

Relacione esta operación con `GPIO_OUT_XOR`.

Leer entradas

```
uint32_t entradas = sio_hw->gpio_in;
bool boton = (entradas & BUTTON_MASK) != 0;
```

Explique qué hace `entradas & BUTTON_MASK`.

Configurar dirección

```
sio_hw->gpio_oe_set = LED_MASK;
sio_hw->gpio_oe_clr = BUTTON_MASK;
```

Explique por qué una instrucción habilita una salida y la otra la deshabilita.

Parte 7. ¿Qué es sio_hw?

```
#include "hardware/structs/sio.h"
```

Abra el archivo correspondiente dentro del Pico SDK e identifique sio_hw_t y la definición de sio_hw. Relacione estos elementos con SIO_BASE = 0xD0000000 y con los campos gpio_in, gpio_out y gpio_oe.

El objetivo no es memorizar la estructura completa, sino comprender que sus campos corresponden a registros hardware mapeados en memoria.

Parte 8. Comprender el monitor de comandos

```
typedef struct {
    const char *name;
    const char *help;
    command_handler_t handler;
} command_t;
```

Observe la tabla commands[]. Interprete la secuencia “nombre escrito -> búsqueda del comando -> función asociada -> operación sobre hardware”.

```
"on" -> cmd_on() -> sio_hw->gpio_set -> GPIO_OUT_SET -> GPIO25 -> LED
```

No se requiere modificar el parser en esta práctica.

4. Modificación obligatoria: comando status

Agregue un nuevo comando llamado status. Debe mostrar únicamente el estado del LED en GPIO25 y del botón en GPIO13.

```
RP2040> status
```

```
GPIO25 - LED : 1
```

```
GPIO13 - BOTON : 0
```

Al presionar el botón, el segundo valor deberá cambiar a 1. La solución debe utilizar registros del SIO y máscaras de bits; no se deberá resolver exclusivamente con gpio_get(). El comando status también debe aparecer al ejecutar help.

Evidencia 4. Captura de status funcionando con el botón libre y presionado.

Evidencia 5. Fragmento del código de cmd_status() y su incorporación a la tabla de comandos.

5. Hoja de evidencias

La entrega será una hoja de evidencias breve. Puede incluir fotografías o capturas directamente dentro del documento; no se requiere un reporte tradicional.

Evidencia	Contenido
1	Monitor ejecutándose y comando help.
2	GPIO_OUT con LED apagado y encendido.
3	GPIO13 con botón libre y presionado.
4	Comando status funcionando.
5	Código de cmd_status().
6	Tabla de observaciones de comandos.
7	Respuestas a las preguntas de análisis.

Preguntas de análisis

1. ¿Qué representa sio_hw?
2. ¿Qué significa $1u \ll 25$?
3. ¿Cuál es la diferencia entre GPIO_IN, GPIO_OUT y GPIO_OE?
4. ¿Qué hace gpio_set?
5. ¿Qué hace gpio_clr?
6. ¿Qué hace gpio_togl?
7. ¿Por qué se utiliza una máscara para modificar un GPIO?
8. ¿Qué hace la operación entradas & BUTTON_MASK?
9. ¿Qué relación existe entre sio_hw_t y los registros mostrados en la hoja de datos del RP2040?
10. Si posteriormente se utiliza ADC, TIMER o UART, ¿qué idea de esta práctica espera volver a encontrar?

6. Escala estimativa

Criterio	Excelente	Satisfactorio	Insuficiente	Valor
Compilación y ejecución	Proyecto funciona correctamente en la Pico.	Funciona con pequeñas dificultades.	No logra ejecutarlo.	1
Exploración de comandos	Prueba y documenta todos los comandos.	Falta alguna evidencia.	Exploración mínima.	1
Comprensión de SIO	Distingue correctamente IN, OUT y OE.	Presenta alguna confusión.	No distingue los registros.	2
Máscaras de bits	Explica y utiliza correctamente $1u \ll n$.	Comprensión parcial.	No comprende su función.	1
Análisis del código	Relaciona código, estructura y registros.	Relación parcial.	Descripción superficial.	1
Comando status	Funciona correctamente usando SIO.	Funciona parcialmente.	No funciona o usa solo funciones de alto nivel.	2
Hoja de evidencias	Completa, clara y ordenada.	Omisiones menores.	Evidencia insuficiente.	1
Preguntas de análisis	Respuestas correctas y razonadas.	Parcialmente correctas.	Incompletas.	1
TOTAL				10

7. Distribución sugerida del tiempo

Tiempo	Actividad
Hora 1	Compilar, cargar y explorar libremente el monitor.
Hora 2	Experimentar con GPIO_IN, GPIO_OUT, GPIO_OE y máscaras.
Hora 3	Revisar el código y relacionarlo con sio_hw_t y el datasheet.
Hora 4	Desarrollar status, probarlo y completar la hoja de evidencias.

Secuencia didáctica: usar -> experimentar -> observar -> analizar -> modificar -> demostrar.